Rapport Projet Machine Learning

CyberGuard MLOps

Plateforme MLOps de détection d'intrusions IoT

CICIoT2023 | FastAPI | MLflow | DVC | Airflow | Docker | Grafana | Streamlit SOC

Une chaîne MLOps complète pour entraîner, valider, déployer et surveiller un modèle de détection d'intrusions dans un contexte SOC moderne.

Informations

Filière	CI2 CYBER
Module	Machine Learning
Professeur	LAZAAR MOHAMED
Année	2025–2026

Réalisé par

AICH FADI
YOUSSOUF KEITA
LEKOGHO OSSIE CELESTIN CALVINE KILLIAN
MAALLEM MEHDI

Mai 2026 | Rapport 20+ pages

Résumé

CyberGuard MLOps est une plateforme MLOps complète pour la détection d'intrusions dans des flux réseau IoT. Le projet utilise un échantillon réel et reproductible du dataset CICIoT2023, publié en 2023 par le Canadian Institute for Cybersecurity de l'University of New Brunswick. La solution couvre tout le cycle de vie d'un modèle : ingestion, validation, expérimentation, tracking, registry, orchestration, API, conteneurisation, monitoring, drift et interface analyste SOC.

Le modèle final retenu est un **Gradient Boosting**. Il atteint un F1-score de **0,941** et un ROC-AUC de **0,956**. Le résultat n'est pas présenté comme un simple score de notebook : il est intégré dans une chaîne industrielle avec `DVC`, `Great Expectations`, `MLflow`, `FastAPI`, `Docker`, `Prometheus`, `Grafana`, `Airflow`, `GitHub Actions` et `Streamlit`.

L'objectif métier est d'aider un analyste SOC à prioriser les flux suspects. L'interface Streamlit expose les KPI, les cartes, le ranking des pays sources, la file d'événements, le live scoring et les preuves modèle. Le dashboard Grafana apporte l'observabilité technique et SOC : disponibilité API, débit de prédiction, ratio d'attaques, latence, sévérité, pays sources, serveurs et drift. Le rapport documente également les commandes de reproduction afin qu'un évaluateur puisse relancer le projet.

Remerciements

Nous remercions notre professeur LAZAAR MOHAMED pour le cadrage du projet et pour l'exigence d'une démarche MLOps complète. Ce travail a été conçu comme une démonstration professionnelle : il privilégie la reproductibilité, la qualité logicielle, la traçabilité et l'utilisabilité par un analyste.

Table des matières

Résumé **i**

Remerciements **ii**

1 Introduction **1**
1.1 Contexte 1
1.2 Problème métier 1
1.3 Objectifs 1
1.4 Métriques de succès 2
1.5 Pourquoi le MLOps est nécessaire 2
1.6 Positionnement de CyberGuard 2

2 Présentation des bases de données **4**
2.1 Origine du dataset 4
2.2 Ingestion 4
2.3 EDA et problèmes identifiés 5
2.4 Validation des données 5
2.5 Versionnage avec DVC 6

3 Architecture MLOps globale **9**
3.1 Vue d'ensemble 9
3.2 Couverture des huit phases 11
3.3 Choix d'architecture 11

4 Algorithmes et pipeline technique **12**
4.1 Features 12
4.2 Prétraitement 12
4.3 Modèles comparés 12
4.4 Choix final 13
4.5 Bugs ML classiques évités 14
4.6 Commandes du pipeline modèle 14

5 Validation, qualité et TDD **15**
5.1 Great Expectations 15
5.2 Qualité automatique 15
5.3 Commandes qualité 16
5.4 TDD adapté au ML 16

6 Tracking et registry **17**
6.1 MLflow Tracking . . . 17
6.2 Model Registry . . . 18
6.3 Model card . . . 18

7 Orchestration et CI/CD **19**
7.1 Airflow . . . 19
7.2 GitHub Actions . . . 20

8 Déploiement, serving et monitoring **22**
8.1 FastAPI . . . 22
8.2 Docker Compose . . . 23
8.3 Prometheus . . . 24
8.4 Grafana SOC Command Center . . . 25
8.5 Drift avec Evidently . . . 25
8.6 Commandes de monitoring . . . 27

9 Interface SOC et résultats **28**
9.1 Interface Streamlit . . . 28
9.2 Structure du projet . . . 30
9.3 Résultats principaux . . . 30

10 Guide de reproduction complet **32**
10.1 Cloner le projet . . . 32
10.2 Créer le venv Python 3.11 . . . 32
10.3 Relancer les artefacts ML . . . 32
10.4 Lancer tous les services . . . 32
10.5 Générer du trafic . . . 32
10.6 URLs . . . 33
10.7 Compilation du rapport . . . 33

11 Conclusion **34**
11.1 Bilan . . . 34
11.2 Difficultés . . . 34
11.3 Perspectives . . . 34

A Inventaire des captures **35**

B Commandes détaillées du projet **37**
B.1 Vérification environnement . . . 37
B.2 Arrêt et redémarrage . . . 37
B.3 Tests API . . . 37
B.4 GitHub . . . 37

C Références **38**

Table des figures

2.1 Ingestion et validation du dataset CICIoT2023. 5
2.2 Pipeline DVC et métriques suivies. 7

3.1 Architecture MLOps détaillée de CyberGuard. 10

4.1 Entraînement et métriques finales. 13

5.1 Tests et contrôles qualité. 16

6.1 MLflow Tracking et registre modèle. 17
6.2 Détails d'un run MLflow : paramètres, métriques et artefacts. 17

7.1 DAG Airflow exécuté avec succès. 19
7.2 Vue graphe Airflow avec dépendances. 20
7.3 GitHub Actions avec workflow réussi. 20
7.4 Dépôt GitHub CyberGuard MLOps. 21

8.1 Documentation interactive FastAPI. 22
8.2 Exemple de réponse de prédiction. 23
8.3 Services CyberGuard dans Docker Desktop. 24
8.4 État Docker Compose dans le terminal. 24
8.5 Prometheus targets actifs. 25
8.6 Dashboard Grafana CyberGuard CICIoT2023 SOC Command Center. 25
8.7 Rapport HTML Evidently. 26
8.8 Rapport SOC HTML. 27

9.1 Vue globale CyberGuard SOC Analyst Workbench. 28
9.2 Live scoring et export d'incident. 29
9.3 File Analyst Queue. 29
9.4 Onglet Model Evidence. 29
9.5 Structure du projet dans VS Code. 30

A.1 Versions principales de l'environnement. 36

Liste des tableaux

1.1 Métriques métier, ML et techniques 2

2.1 Fiche descriptive du dataset 4
2.2 Validation des données 6
2.3 Stages DVC 8

3.1 Correspondance entre le sujet et l'implémentation 11

4.1 Familles de features 12
4.2 Comparaison des métriques ML 13
4.3 Risques ML et protections 14

5.1 Résultat Great Expectations 15

7.1 Étapes GitHub Actions 21

8.1 Services Docker 24
8.2 Résumé du drift 26

9.1 Synthèse des preuves du rendu 31

10.1 URLs de démonstration 33

A.1 Captures intégrées au rapport 35

1 Introduction

1.1 Contexte

Les environnements IoT modernes produisent une grande quantité de flux réseau. Dans un contexte de cybersécurité, ces flux peuvent contenir des attaques par déni de service, reconnaissance, brute force, spoofing, Mirai ou attaques web. Le problème n'est pas seulement de construire un modèle capable de classer un flux. Le vrai défi est de rendre ce modèle exploitable, contrôlable et surveillé.

Un modèle non industrialisé peut donner un bon score localement puis échouer en production pour des raisons simples : dépendances différentes, données invalides, fuite d'information, absence de monitoring, dérive des distributions ou impossibilité de retrouver la version exacte du modèle. C'est précisément le rôle du MLOps : fiabiliser le cycle complet du modèle, depuis les données jusqu'à la supervision en production.

1.2 Problème métier

Le problème métier traité est la **priorisation SOC de flux réseau IoT**. Pour chaque flux, le système doit produire :

- une prédiction binaire : bénin ou attaque ;
- une probabilité d'attaque ;
- un niveau de risque ;
- des métriques de monitoring exploitables dans Grafana ;
- une preuve consultable par un analyste.

Question centrale

Comment construire une plateforme MLOps capable de détecter des intrusions IoT, de tracer toutes les expériences, de servir le modèle par API, de surveiller la production et de fournir une interface analyste claire ?

1.3 Objectifs

Les objectifs du projet sont :

- utiliser un dataset réel, public et récent ;
- construire un pipeline reproductible avec DVC ;
- valider les données avec une approche Great Expectations ;
- comparer plusieurs algorithmes avec MLflow ;

— enregistrer le meilleur modèle dans un registre ;
— exposer le modèle avec FastAPI ;
— déployer les composants avec Docker Compose ;
— monitorer l'API avec Prometheus et Grafana ;
— détecter le drift avec Evidently ;
— orchestrer le pipeline avec Airflow ;
— automatiser les tests avec GitHub Actions ;
— produire un rapport professionnel en LaTeX.

1.4 Métriques de succès

Table 1.1 – Métriques métier, ML et techniques

Catégorie	Métriques retenues
Métier	Nombre d'alertes priorisées, visualisation des pays sources, distribution des sévérités, capacité à exporter une note d'incident.
ML	Accuracy, précision, rappel, F1-score, ROC-AUC. Le F1-score est la métrique principale.
Technique	Disponibilité API, débit de prédiction, ratio d'attaques prédites, latence p95, état Prometheus/Grafana.
Gouvernance	Reproductibilité DVC, tracking MLflow, CI GitHub Actions, model card, preuves de validation.

1.5 Pourquoi le MLOps est nécessaire

Dans un projet cyber, un modèle isolé n'est pas suffisant. Il faut savoir quelle donnée l'a entraîné, quels paramètres ont été utilisés, quels tests ont été exécutés, quelle image Docker le sert, si l'API répond, si les distributions dérivent et comment un analyste peut interpréter le résultat. CyberGuard répond donc à la logique suivante :

1. **Reproduire** : mêmes données, mêmes scripts, mêmes artefacts ;
2. **Valider** : données, code, modèle et API ;
3. **Déployer** : API et UI dans des conteneurs ;
4. **Observer** : métriques techniques et ML ;
5. **Améliorer** : drift, feedback et réentraînement.

1.6 Positionnement de CyberGuard

CyberGuard est positionné comme une preuve de concept avancée. Il ne prétend pas remplacer un SIEM, un IDS ou une plateforme SOAR, mais il montre comment un modèle ML peut être intégré dans une chaîne SOC réaliste. La valeur du projet est donc double :

d'un côté, un classifieur entraîné sur des flux IoT ; de l'autre, une infrastructure permettant de le tester, l'exposer, le monitorer et documenter ses limites.

Cette distinction est essentielle dans le rapport. Un projet ML académique classique s'arrêterait au tableau de métriques. Ici, chaque résultat est relié à une preuve d'exploitation : API documentée, dashboard Grafana, drift report, run MLflow, DAG Airflow, workflow GitHub Actions et interface Streamlit. Le rapport insiste donc sur la capacité à **opérer** le modèle, pas seulement à l'entraîner.

2 Présentation des bases de données

2.1 Origine du dataset

Le projet utilise **CICIoT2023**, dataset public publié en 2023 par le Canadian Institute for Cybersecurity de l'University of New Brunswick. L'article associé décrit un benchmark d'attaques à grande échelle dans un environnement IoT, avec 105 appareils et 33 attaques réparties dans plusieurs familles [1], [2].

Le dataset complet est volumineux. Pour garantir une exécution reproductible sur une machine locale, le projet ingère un échantillon de 3 000 lignes. Ce choix est volontaire : il permet de démontrer la démarche MLOps complète sans transformer le rendu en problème de stockage massif.

Table 2.1 – Fiche descriptive du dataset

Champ	Description
Dataset	CICIoT2023
Année	2023
Origine	Canadian Institute for Cybersecurity, University of New Brunswick
Type de données	Flux réseau IoT
Problème	Classification binaire attaque/bénin
Taille utilisée	3 000 lignes validées
Fenêtres	2 100 lignes de référence et 900 lignes courantes pour le drift
Target	`is_attack`

2.2 Ingestion

L'ingestion est réalisée par le module `cyberguard_ml.pipeline.ingest_ciciot2023`. Il télécharge ou récupère les données, normalise les colonnes, construit la cible binaire et crée des enrichissements SOC pour les dashboards.

```
$env:PYTHONPATH='src'
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.ingest_ciciot2023 --rows
    3000 --page-size 100 --offset 0
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   ···   powershell

(.venv) PS C:\Users\fadia\OneDrive\Bureau\Projet ML> $env:PYTHONPATH='src'
>>    .\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.ingest_ciciot2023 --rows 3000 --page-size
100 --offset 0
>>    .\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.validate_data
{
  "passed": true,
  "row_count": 3000,
  "attack_rate": 0.8433333333333334,
  "errors": [],
  "warnings": []
}
(.venv) PS C:\Users\fadia\OneDrive\Bureau\Projet ML>
```

Figure 2.1 – Ingestion et validation du dataset CICIoT2023.

Le script d'ingestion a été conçu pour être relançable. Il accepte un nombre de lignes, une taille de page et un offset afin de pouvoir changer la taille de l'échantillon sans modifier le code. Cette approche évite de figer la donnée dans le projet : si un autre échantillon est souhaité, il suffit de relancer l'ingestion puis DVC détectera les artefacts à reconstruire.

2.3 EDA et problèmes identifiés

L'EDA du projet est orientée MLOps : elle sert à identifier les risques qui peuvent casser une chaîne d'entraînement ou fausser le modèle.

— **Déséquilibre de classes** : l'échantillon contient 84,33 % d'attaques. Le split doit donc être stratifié.

— **Variables très dispersées** : certaines colonnes, notamment `variance`, peuvent changer fortement entre les fenêtres.

— **Risque de leakage** : les champs `source_country`, `source_ip`, `destination_server` et `severity` sont utiles pour le SOC, mais exclus des features.

— **Drift potentiel** : la comparaison référence/courant détecte un drift sur `variance`.

2.4 Validation des données

La validation produit `reports/data_validation.json`. Elle vérifie le nombre de lignes, la cible, les colonnes nécessaires, les erreurs bloquantes et les avertissements. Le résultat courant est :

Table 2.2 – Validation des données

Indicateur	Valeur
Validation globale	Réussie
Nombre de lignes	3 000
Taux d'attaque	84,33 %
Erreurs	0
Avertissements	0

```
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.validate_data
Get-Content .\reports\data_validation.json
```

Cette validation joue le rôle de garde-fou avant l'entraînement. Si le schéma change, si la cible disparaît ou si une colonne essentielle est invalide, le pipeline doit échouer tôt. Dans un contexte MLOps, échouer tôt est préférable à entraîner silencieusement un modèle incohérent.

2.5 Versionnage avec DVC

DVC formalise les dépendances, sorties et métriques du pipeline. Le fichier `dvc.yaml` agit comme une carte de reproductibilité : si une dépendance change, DVC sait quelle étape relancer [3].

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

(.venv) PS C:\Users\fadia\OneDrive\Bureau\Projet ML> .\.venv\Scripts\dvc.exe dag
>>    .\.venv\Scripts\dvc.exe metrics show
>>    .\.venv\Scripts\dvc.exe status
WARNING: Unable to find `less` in the PATH. Check out <https://man.dvc.org/pipeline/show> for more info.
                +---------------+
                | generate_data |
                +---------------+
                ***             **
              **                  ***
            **                       **
+--------------+               +---------------+
| drift_report |               | validate_data |
+--------------+               +---------------+
                               **              **
                            ***                 ***
                          **                       **
                +-------------+               +--------------------+
                | train_model |               | great_expectations |
                +-------------+               +--------------------+
Path                          attack_rate    passed    row_count
reports\data_validation.json  0.84333        True      3000
models\metrics.json           -              -         -
generate_data:
        changed outs:
                modified:           data\raw\network_flows.csv
                modified:           data\processed\reference_window.csv
                modified:           data\processed\current_window.csv
validate_data:
        changed deps:
                modified:           data\raw\network_flows.csv
        changed outs:
                modified:           data\processed\network_flows_validated.csv
train_model:
        changed deps:
                modified:           data\processed\network_flows_validated.csv
                modified:           src\cyberguard_ml\pipeline\train_model.py
drift_report:
        changed deps:
                modified:           data\processed\reference_window.csv
                modified:           data\processed\current_window.csv
great_expectations:
        changed deps:
                modified:           data\processed\network_flows_validated.csv
(.venv) PS C:\Users\fadia\OneDrive\Bureau\Projet ML>
```

Figure 2.2 – Pipeline DVC et métriques suivies.

```
.\.venv\Scripts\dvc.exe repro
.\.venv\Scripts\dvc.exe dag
.\.venv\Scripts\dvc.exe metrics show
.\.venv\Scripts\dvc.exe status
```

Table 2.3 – Stages DVC

Stage	**Rôle**
`generate_data`	Ingestion CICIoT2023 et création des fenêtres de référence/courante.
`validate_data`	Validation du schéma et export du dataset validé.
`train_model`	Entraînement, comparaison des modèles et export du meilleur artefact.
`drift_report`	Rapport de drift Evidently.
`great_expectations`	Suite et résultat de validation formalisés.

3 Architecture MLOps globale

3.1 Vue d'ensemble

L'architecture CyberGuard suit les huit phases demandées dans le sujet : conception, données, développement modèle, automatisation, validation, déploiement, monitoring et gouvernance. La figure suivante remplace le schéma générique par le process flow réel du projet.

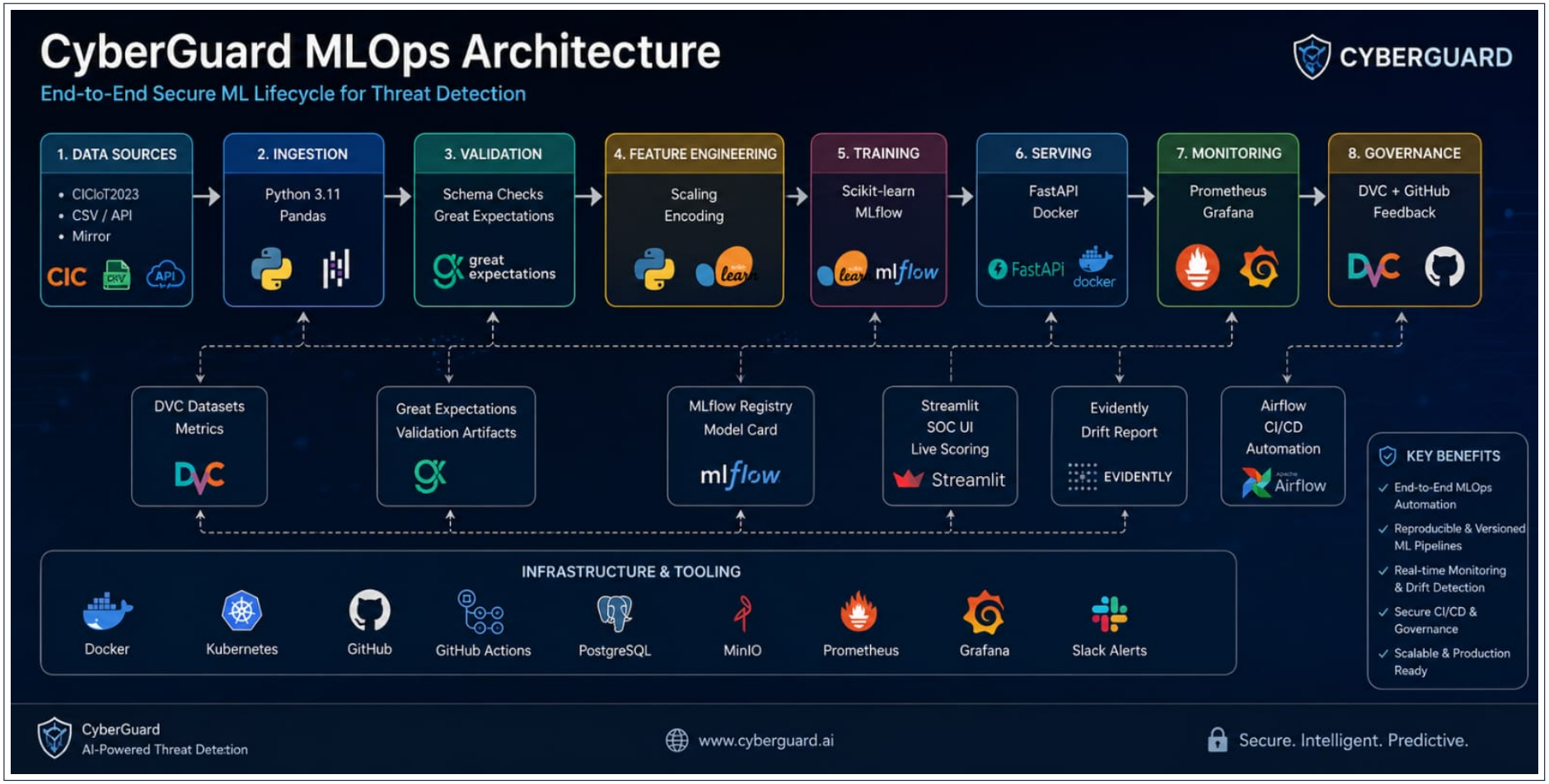

Figure 3.1 – Architecture MLOps détaillée de CyberGuard.

La figure 3.1 résume le fonctionnement opérationnel du projet. Les données CICIoT2023 entrent dans le pipeline d'ingestion, sont validées, transformées puis utilisées pour entraîner plusieurs modèles. Le meilleur modèle est tracé dans MLflow et servi via FastAPI. En parallèle, Prometheus collecte les métriques de l'API, Grafana les transforme en dashboard SOC, Evidently analyse la dérive et Airflow orchestre la chaîne. Cette architecture est volontairement hybride : elle montre à la fois le chemin data science, le chemin déploiement et le chemin observabilité.

3.2 Couverture des huit phases

Table 3.1 – Correspondance entre le sujet et l'implémentation

Phase	Implémentation
Conception	Définition d'un cas SOC : classifier des flux IoT et aider l'analyste à prioriser.
Données	Ingestion CICIoT2023, validation, nettoyage, enrichissement SOC et DVC.
Modèle	Comparaison Logistic Regression, Random Forest, Gradient Boosting et sélection par F1-score.
Automatisation	DVC pour le pipeline local et Airflow pour l'orchestration.
Validation	Pytest, Great Expectations, model card, registry MLflow et qualité code.
Déploiement	FastAPI, Docker Compose, inference temps réel, batch et locale.
Monitoring	Prometheus, Grafana, Evidently, SOC threat report.
Gouvernance	GitHub Actions, documentation, versionning, feedback et ré-entraînement.

3.3 Choix d'architecture

Les choix techniques privilégient la reproductibilité et la démonstrabilité :

- `Python 3.11` : version stable et cohérente avec Airflow/Docker ;
- `scikit-learn` : modèles robustes, rapides et adaptés à un rapport académique ;
- `MLflow` : tracking, artefacts et registry ;
- `DVC` : pipeline et métriques versionnables ;
- `FastAPI` : API rapide avec documentation OpenAPI ;
- `Docker Compose` : reproductibilité multi-services ;
- `Prometheus/Grafana` : monitoring technique et SOC ;
- `Streamlit` : interface analyste rapide et interactive.

4 Algorithmes et pipeline technique

4.1 Features

Les features utilisées représentent des propriétés réseau : protocoles, flags TCP, taille de paquets, temporalité, taux et agrégats statistiques. Les variables d'enrichissement SOC sont exclues pour éviter une fuite de données.

Table 4.1 – Familles de features

Famille	**Exemples**
Protocoles	`protocol_type`, `tcp`, `udp`, `icmp`, `http`, `https`, `dns`.
Flags TCP	`fin_flag_number`, `syn_flag_number`, `rst_flag_number`, `ack_flag_number`.
Statistiques paquets	`min_packet_size`, `max_packet_size`, `avg_packet_size`, `std_packet_size`.
Temporalité	`iat`, `rate`, `time_to_live`.
Agrégats	`tot_sum`, `tot_size`, `packet_number`, `variance`.

4.2 Prétraitement

Le pipeline ML applique une séparation explicite entre les colonnes numériques et catégorielles. Les colonnes numériques sont standardisées et les colonnes catégorielles encodées. Cette organisation garantit que l'entraînement et l'inférence utilisent la même transformation.

Le prétraitement est intégré au modèle sauvegardé. Cela évite un bug fréquent : entraîner avec une transformation locale puis servir en production avec une transformation différente. Le même artefact `models/cyberguard_model.joblib` contient donc la logique nécessaire pour appliquer les transformations attendues au moment de l'inférence.

4.3 Modèles comparés

— **Logistic Regression** : baseline linéaire, rapide, utile pour vérifier si le signal est exploitable.

— **Random Forest** : modèle d'ensemble robuste, performant sur interactions non linéaires.

— **Gradient Boosting** : modèle final, retenu pour le meilleur F1-score.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                    powershell

2026/05/07 14:56:46 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires ex
ercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary co
de during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://
scikit-learn.org/stable/model_persistence.html
Registered model 'CyberGuard-CICIoT2023-Intrusion-Detector' already exists. Creating a new version of this model...
Created version '3' of model 'CyberGuard-CICIoT2023-Intrusion-Detector'.
[
  {
    "name": "logistic_regression",
    "accuracy": 0.8348484848484848,
    "precision": 1.0,
    "recall": 0.8043087971274686,
    "f1": 0.891542288557214,
    "roc_auc": 0.9381569085426436
  },
  {
    "name": "random_forest",
    "accuracy": 0.8909090909090909,
    "precision": 0.9821073558648111,
    "recall": 0.8868940754039497,
    "f1": 0.9320754716981132,
    "roc_auc": 0.9614090742709731
  },
  {
    "name": "gradient_boosting",
    "accuracy": 0.9030303030303031,
    "precision": 0.9624765478424016,
    "recall": 0.9210053859964094,
    "f1": 0.9412844036697248,
    "roc_auc": 0.9565285597252968
  }
]
(.venv) PS C:\Users\fadia\OneDrive\Bureau\Projet ML>
```

Figure 4.1 – Entraînement et métriques finales.

Table 4.2 – Comparaison des métriques ML

Modèle	Accuracy	Précision	Rappel	F1	ROC-AUC
Logistic Regression	0,835	1,000	0,804	0,892	0,938
Random Forest	0,891	0,982	0,887	0,932	**0,961**
Gradient Boosting	**0,903**	0,962	**0,921**	**0,941**	0,956

4.4 Choix final

Le Gradient Boosting est choisi car il maximise le F1-score. Dans ce contexte, le F1-score est plus adapté que l'accuracy seule : il équilibre la capacité à détecter les attaques et la capacité à éviter trop de faux positifs.

Le Random Forest obtient le meilleur ROC-AUC, mais le Gradient Boosting garde l'avantage sur le F1-score et le rappel. Pour un SOC, le rappel est particulièrement important : une attaque manquée peut être plus coûteuse qu'une alerte à vérifier. Le choix final est donc cohérent avec l'objectif métier de triage défensif.

4.5 Bugs ML classiques évités

Table 4.3 – Risques ML et protections

Risque	Protection mise en place
Data leakage	Exclusion des colonnes SOC enrichies des features.
Split non stratifié	Split stratifié pour conserver la distribution de la cible.
Mauvaise target	Target unique `is_attack`.
Données invalides	Validation schéma/ranges avant entraînement.
Confusion bug logiciel/ML	API health pour logiciel, métriques/drift pour ML.

4.6 Commandes du pipeline modèle

```
$env:PYTHONPATH='src'
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.train_model
Get-Content .\models\metrics.json
Get-Content .\models\model_card.json
```

5 Validation, qualité et TDD

5.1 Great Expectations

Great Expectations formalise des règles de qualité sur les données : colonnes attendues, types, valeurs admissibles et cohérence globale [4]. Le projet génère deux artefacts :

- `reports/great_expectations_suite.json` ;
- `reports/great_expectations_validation.json`.

```
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.great_expectations_check
Get-Content .\reports\great_expectations_validation.json
```

Table 5.1 – Résultat Great Expectations

Indicateur	Valeur
Validation	Réussie
Nombre de lignes	3 000
Taux d'attaque	84,33 %
Erreurs	0
Avertissements	0

5.2 Qualité automatique

La chaîne qualité vérifie le code avant publication :

- `Black` : formatage ;
- `isort` : imports ;
- `Ruff` : linting moderne ;
- `MyPy` : typage statique ;
- `Pytest` : tests unitaires et d'intégration.

```
(.venv) PS C:\Users\fadia\OneDrive\Bureau\Projet ML> .\.venv\Scripts\python.exe -m pytest
>>    .\.venv\Scripts\python.exe -m ruff check src tests dags scripts
>>    .\.venv\Scripts\python.exe -m mypy src
.........
9 passed in 10.48s
All checks passed!
Success: no issues found in 14 source files
(.venv) PS C:\Users\fadia\OneDrive\Bureau\Projet ML>
```

Figure 5.1 – Tests et contrôles qualité.

5.3 Commandes qualité

```
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe -m black --check src tests dags scripts
.\.venv\Scripts\python.exe -m isort --check-only src tests dags scripts
.\.venv\Scripts\python.exe -m ruff check src tests dags scripts
.\.venv\Scripts\python.exe -m mypy src
```

5.4 TDD adapté au ML

Les tests ne se limitent pas à vérifier que les fonctions retournent une valeur. Ils vérifient aussi les invariants du pipeline :

— le dataset validé existe ;

— le modèle peut être chargé ;

— l'API répond ;

— les payloads de prédiction sont corrects ;

— les agrégations de l'interface SOC sont cohérentes.

6 Tracking et registry

6.1 MLflow Tracking

MLflow Tracking enregistre les paramètres, métriques, artefacts et modèles de chaque run [5]. CyberGuard l'utilise pour comparer les candidats et conserver la preuve expérimentale.

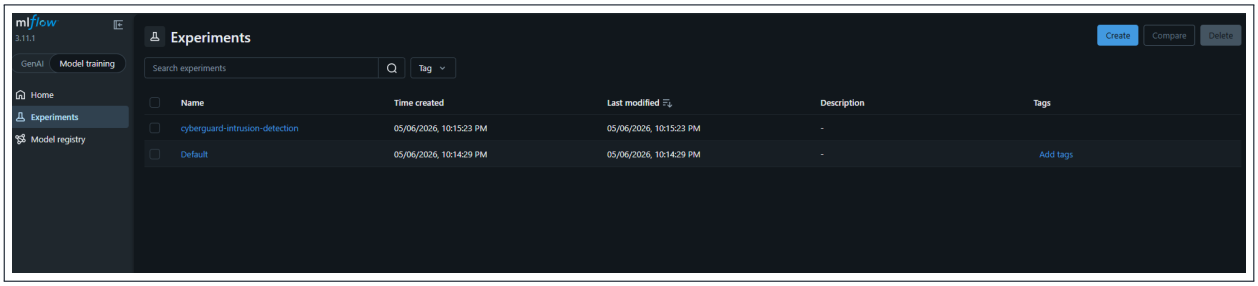

Figure 6.1 – MLflow Tracking et registre modèle.

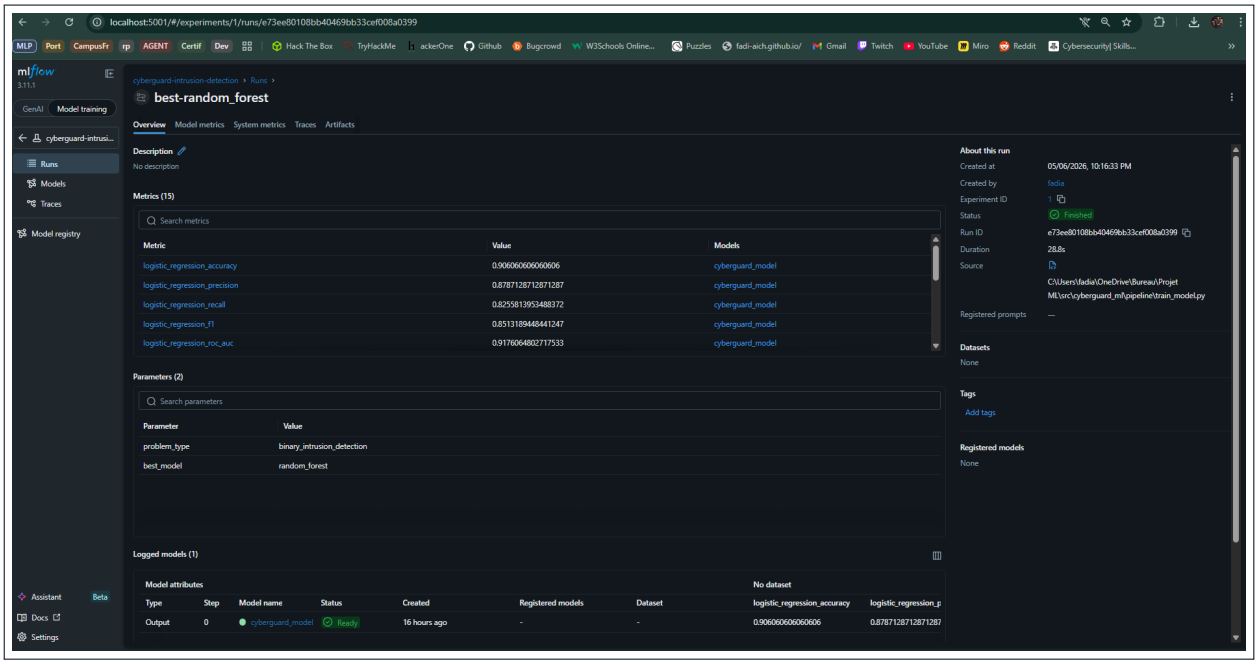

Figure 6.2 – Détails d'un run MLflow : paramètres, métriques et artefacts.

6.2 Model Registry

Le modèle final est enregistré comme `CyberGuard-CICIoT2023-Intrusion-Detector`. Le cycle de vie recommandé est :

1. **Candidate** : modèle entraîné et loggé ;
2. **Staging** : modèle validé par tests et métriques ;
3. **Production** : modèle utilisé par FastAPI ;
4. **Archived** : ancienne version conservée pour audit.

6.3 Model card

Le model card décrit le dataset, l'usage, les limites, le meilleur candidat et les features. Il précise que le modèle est un outil de triage éducatif et non une barrière de sécurité unique.

```
$env:MLFLOW_TRACKING_URI='http://localhost:5001'
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.train_model
Get-Content .\models\model_card.json
```

7 Orchestration et CI/CD

7.1 Airflow

Airflow représente le pipeline sous forme de DAG : chaque tâche a un état, des logs et des dépendances [6]. Cette vue est importante pour le rapport car elle prouve que le pipeline peut être orchestré, pas seulement exécuté à la main.

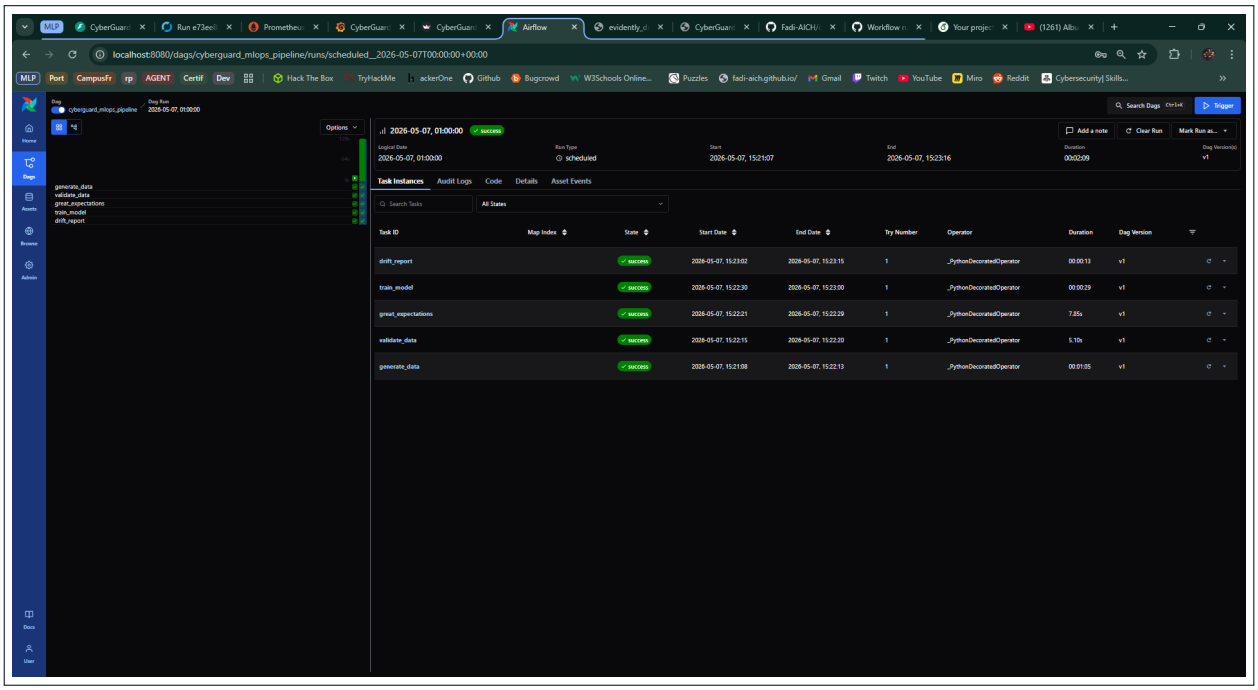

Figure 7.1 – DAG Airflow exécuté avec succès.

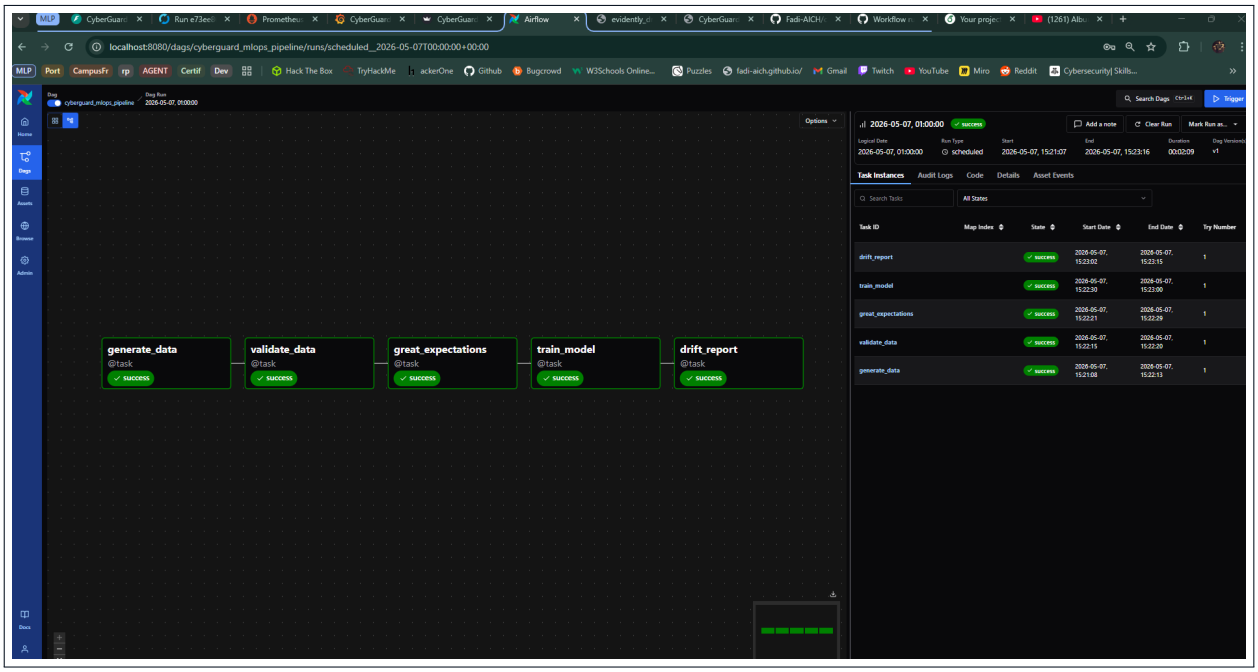

Figure 7.2 – Vue graphe Airflow avec dépendances.

```
docker compose --profile airflow up -d airflow
docker exec cyberguard-airflow bash -lc "airflow dags list"
docker exec cyberguard-airflow bash -lc "airflow dags test
    cyberguard_mlops_pipeline 2026-05-06"
```

7.2 GitHub Actions

GitHub Actions lance un workflow à chaque push, pull request ou déclenchement manuel [7]. Le workflow vérifie le formatage, le linting, le typage, les tests, la génération d'artefacts et les builds Docker.

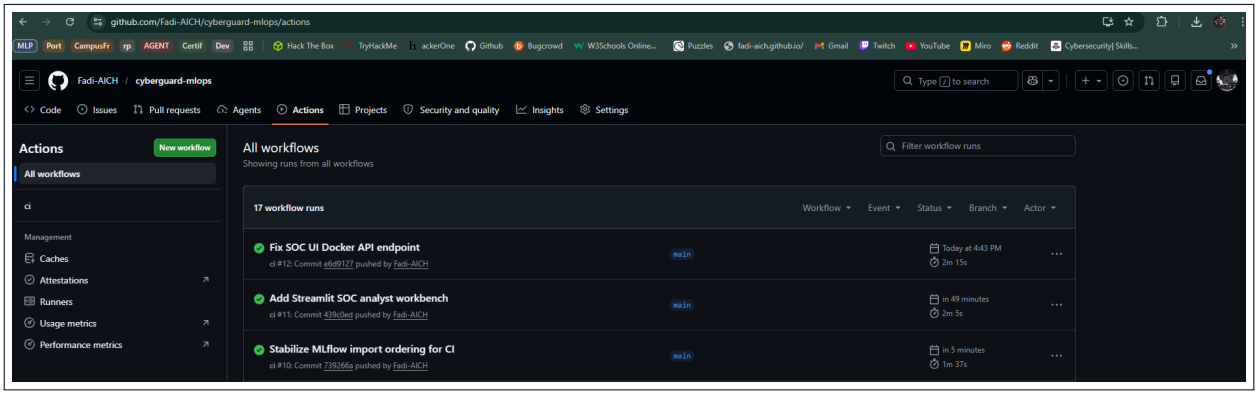

Figure 7.3 – GitHub Actions avec workflow réussi.

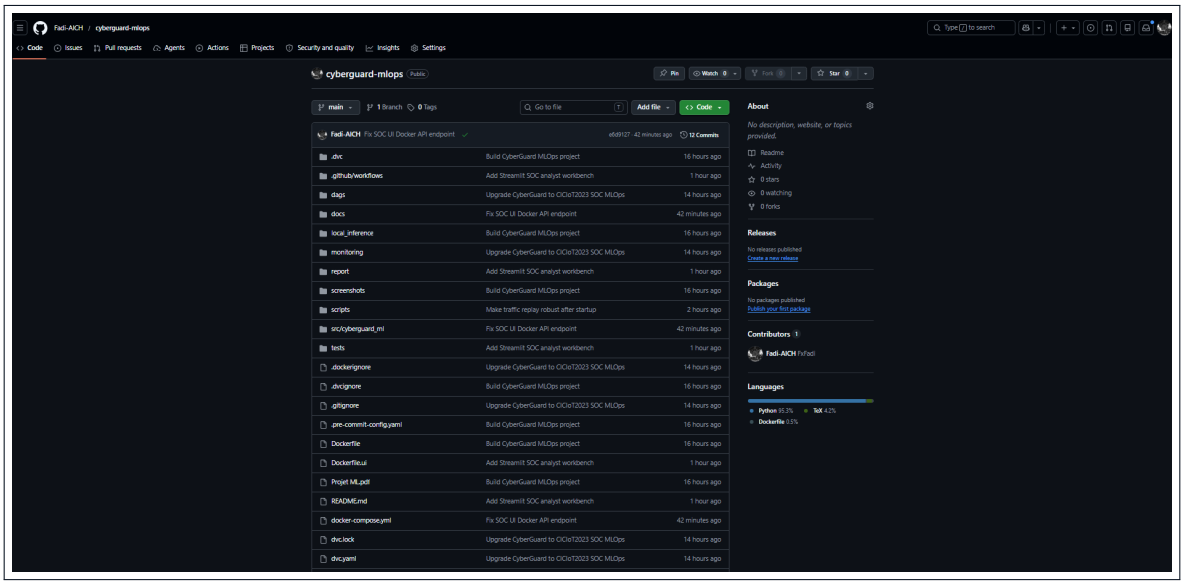

Figure 7.4 – Dépôt GitHub CyberGuard MLOps.

Table 7.1 – Étapes GitHub Actions

Étape	Description
Install	Installe Python 3.11 et les dépendances.
Format/lint	Black, isort, Ruff, MyPy.
Artifacts	Ingestion, validation, entraînement.
Tests	Pytest.
Build API	Construction image Docker FastAPI.
Build SOC UI	Construction image Docker Streamlit.

```
gh workflow run ci.yml --repo Fadi-AICH/cyberguard-mlops
gh run list --repo Fadi-AICH/cyberguard-mlops --limit 5
```

8 Déploiement, serving et monitoring

8.1 FastAPI

FastAPI expose le modèle en REST. Les endpoints principaux sont `/health`, `/predict`, `/batch-predict` et `/metrics`. La documentation Swagger permet de tester l'API directement dans le navigateur.

Le endpoint `/health` sert aux vérifications de disponibilité. Le endpoint `/predict` sert au scoring temps réel. Le endpoint `/batch-predict` permet d'envoyer plusieurs flux en une seule requête. Enfin, `/metrics` expose les compteurs et histogrammes utilisés par Prometheus. Cette séparation rend l'API lisible et facilite les tests de déploiement.

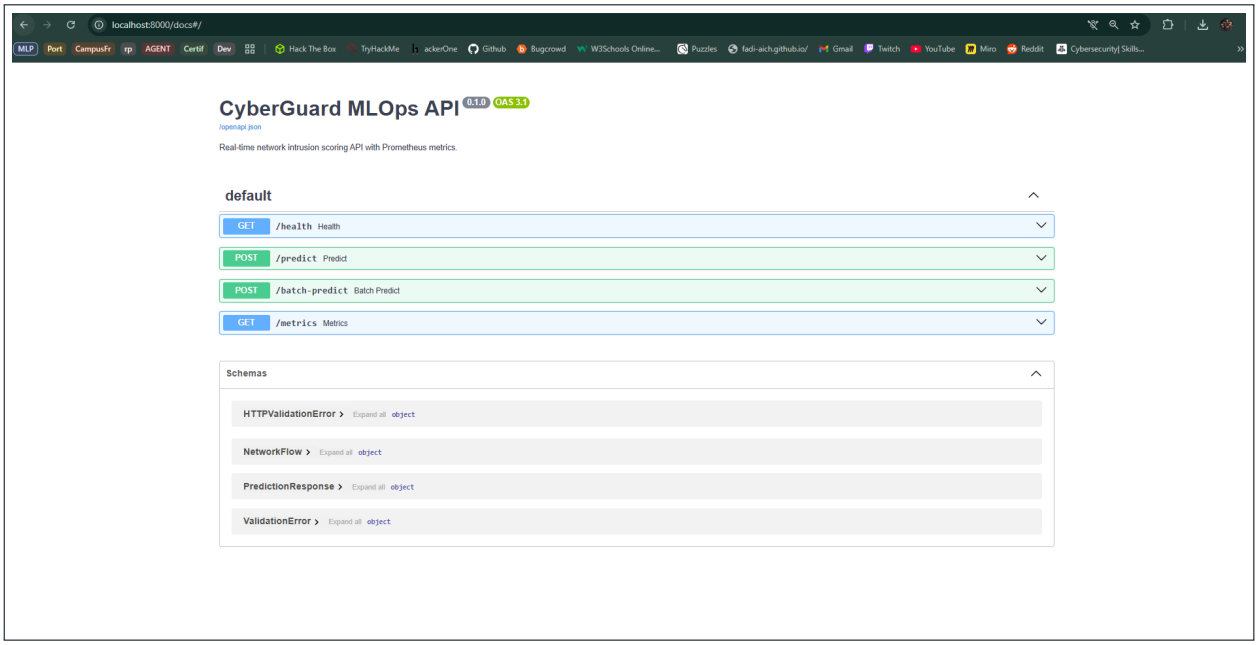

Figure 8.1 – Documentation interactive FastAPI.

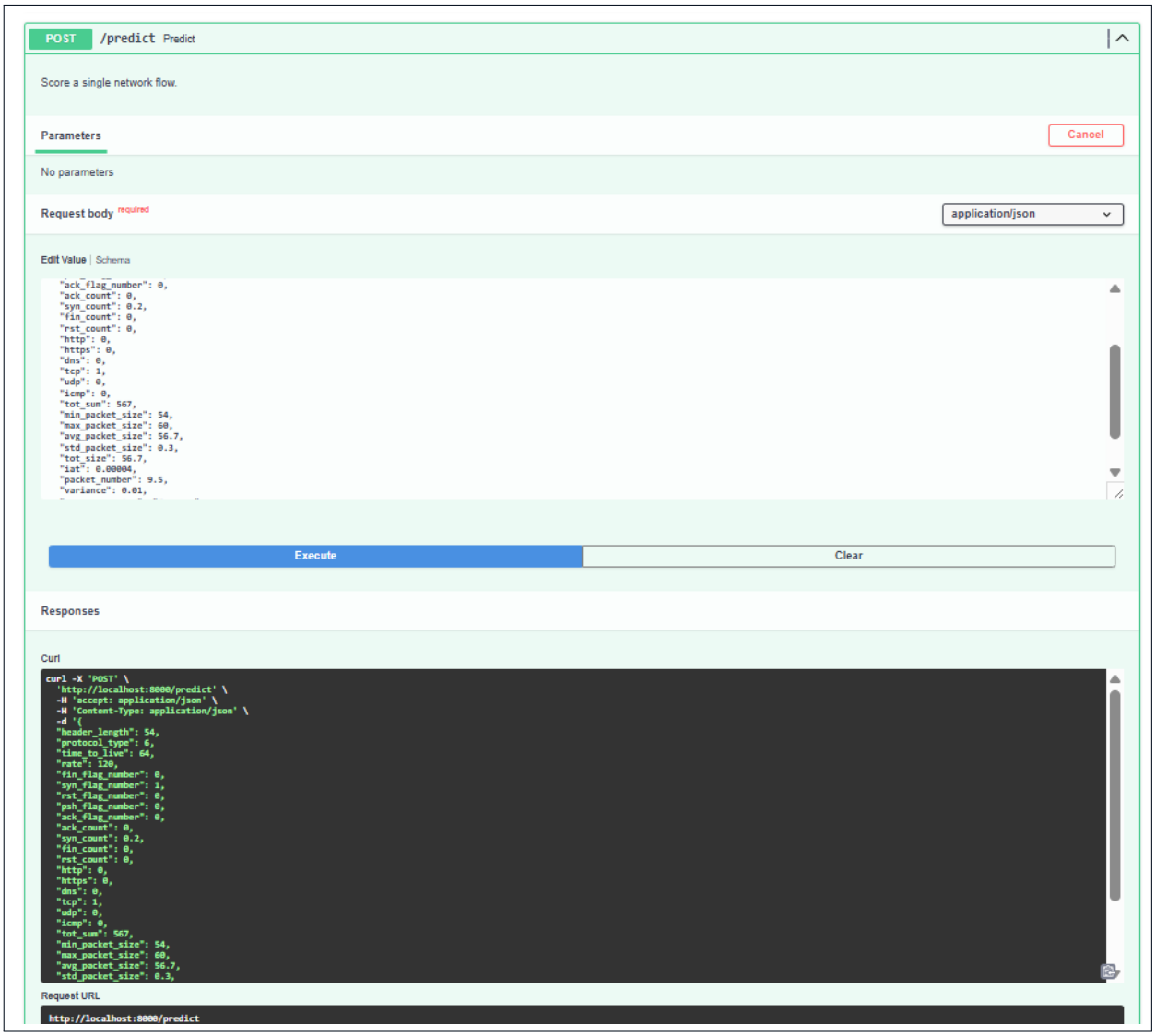

Figure 8.2 – Exemple de réponse de prédiction.

8.2 Docker Compose

Docker Compose lance les services nécessaires au projet. MLflow est exposé sur le port 5001 car le port 5000 peut être occupé sous Windows.

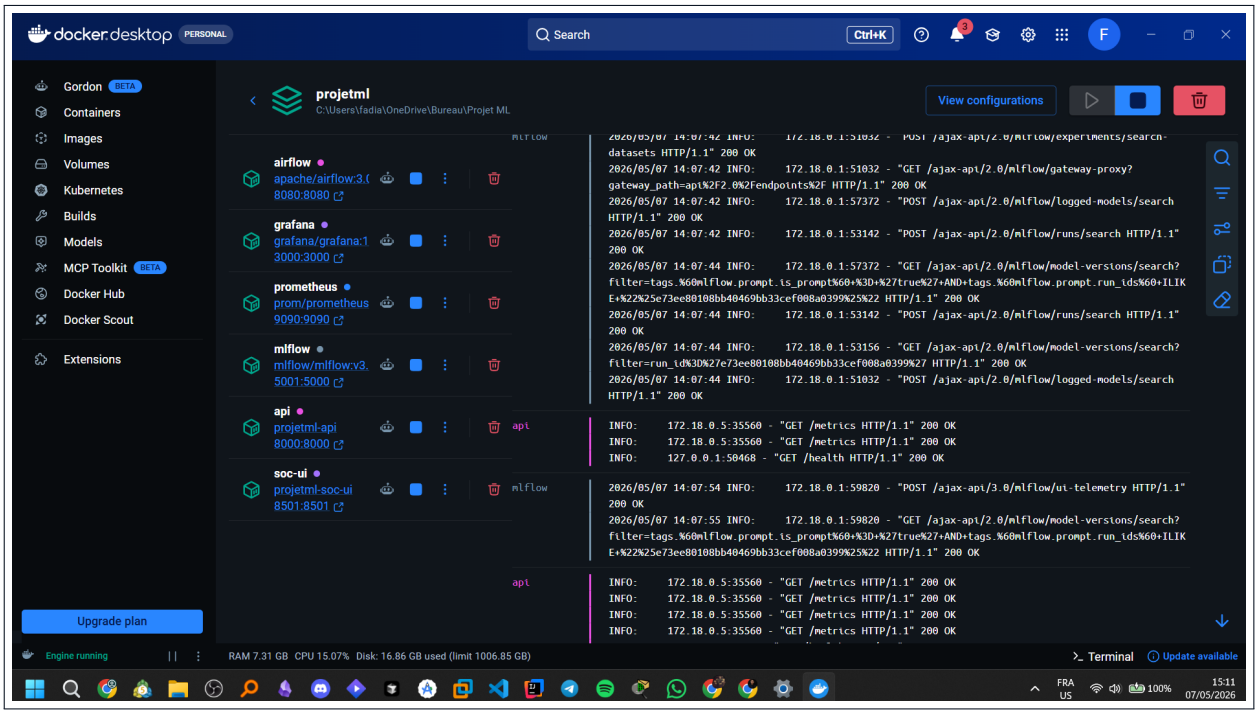

Figure 8.3 – Services CyberGuard dans Docker Desktop.

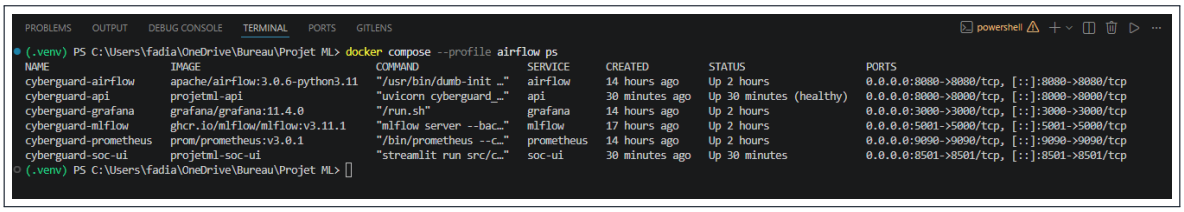

Figure 8.4 – État Docker Compose dans le terminal.

Table 8.1 – Services Docker

Service	Port	Outil	Rôle
api	8000	FastAPI	Serving et métriques.
mlflow	5001	MLflow	Tracking et registry.
prometheus	9090	Prometheus	Scraping des métriques.
grafana	3000	Grafana	Dashboard SOC.
soc-ui	8501	Streamlit	Interface analyste.
airflow	8080	Airflow	Orchestration.

8.3 Prometheus

Prometheus collecte les métriques exposées par l'API : nombre de prédictions, attaques, latence, labels pays/serveur/sévérité et santé du service [8].

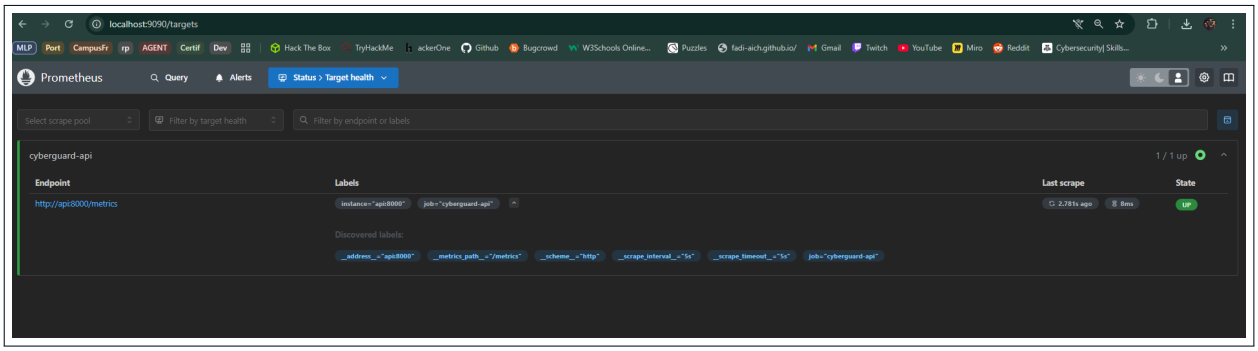

Figure 8.5 – Prometheus targets actifs.

8.4 Grafana SOC Command Center

Le dashboard Grafana a été conçu comme une console SOC : KPI supérieurs, séries temporelles, distribution de sévérité, ranking des attaquants, carte monde et matrice pays/serveur [9].

Le dashboard n'est pas seulement décoratif. Il répond à des questions opérationnelles : l'API est-elle disponible ? Combien de prédictions sont faites ? Quelle part est malveillante ? Quels pays sources dominent les alertes ? Quels serveurs sont touchés ? La latence augmente-t-elle ? Ces éléments permettent de relier le modèle à la supervision de production.

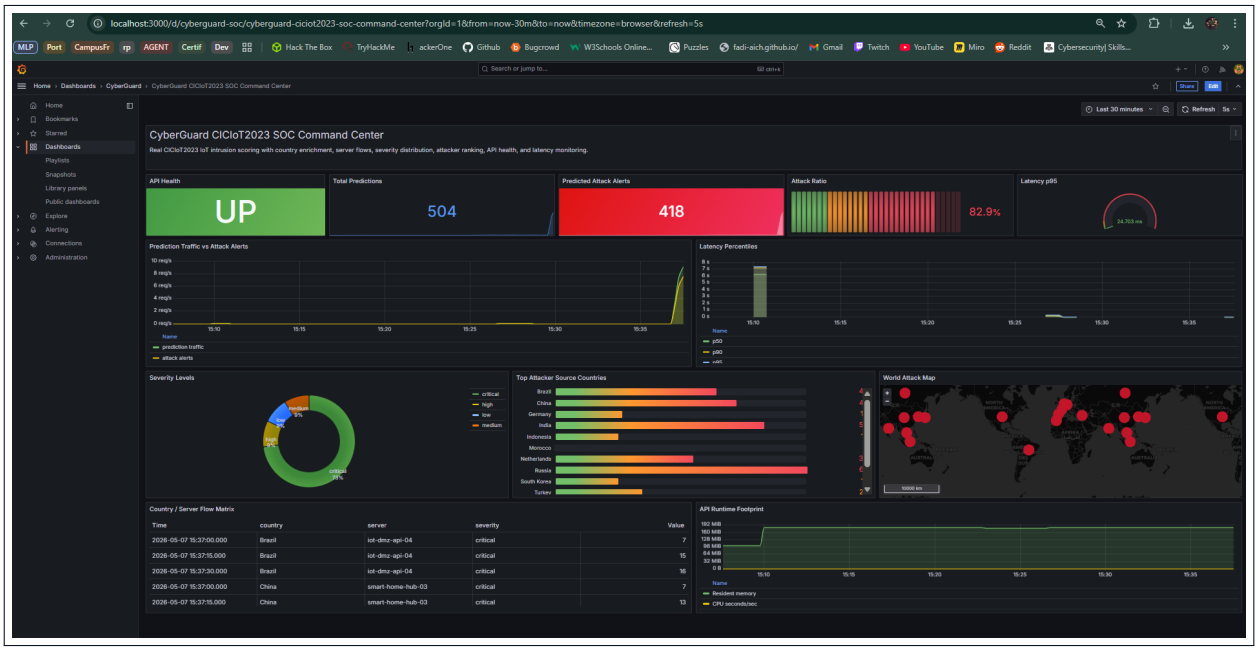

Figure 8.6 – Dashboard Grafana CyberGuard CICIoT2023 SOC Command Center.

8.5 Drift avec Evidently

Evidently compare la fenêtre de référence et la fenêtre courante [10]. Le rapport actuel détecte une dérive sur `variance`.

Table 8.2 – Résumé du drift

Indicateur	Valeur
Drift détecté	Oui
Colonne	`variance`
Score relatif	0,2883
Référence	2 100 lignes
Courant	900 lignes

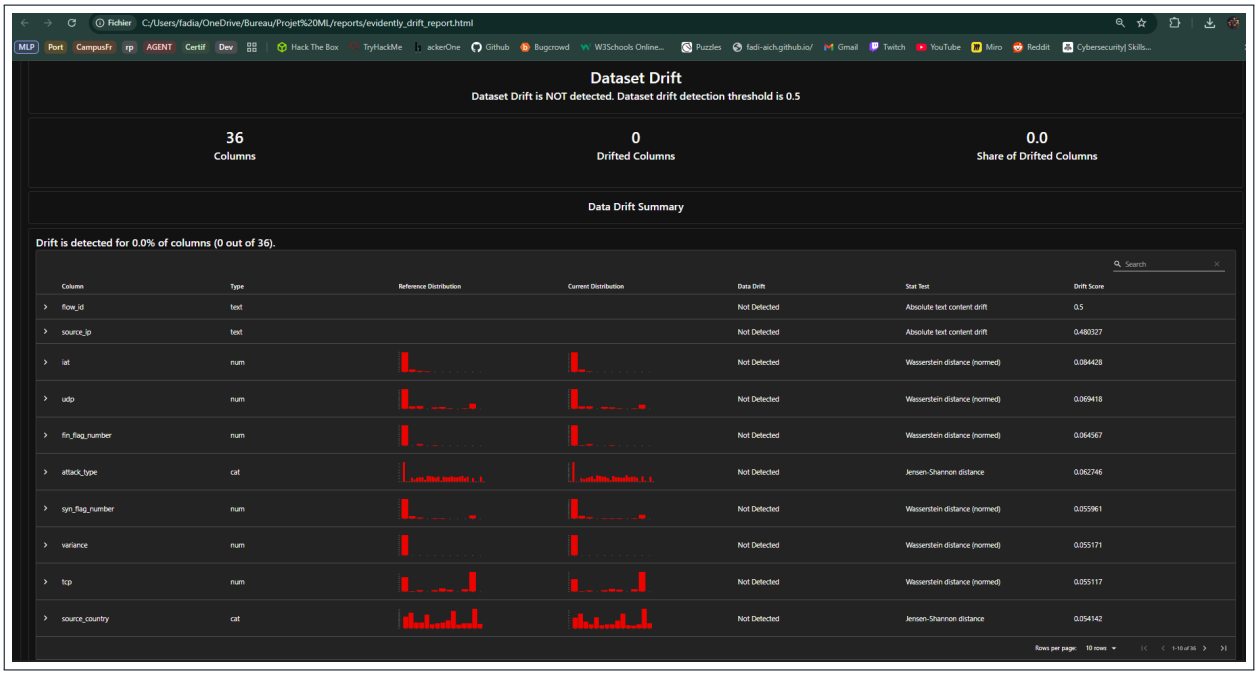

Figure 8.7 – Rapport HTML Evidently.

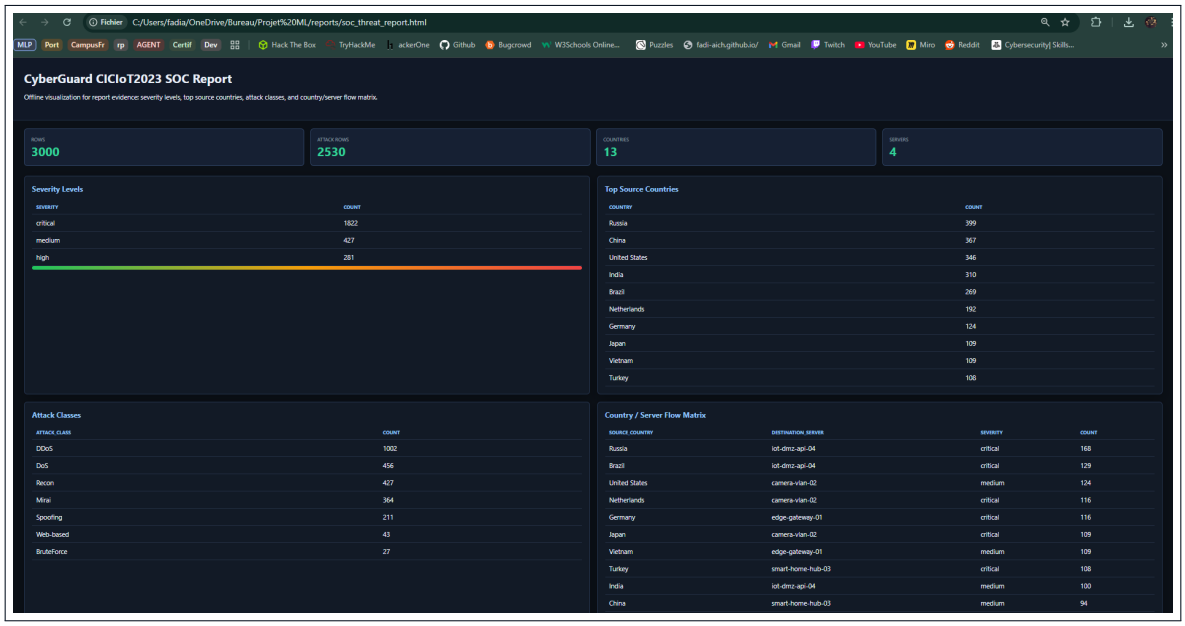

Figure 8.8 – Rapport SOC HTML.

8.6 Commandes de monitoring

```
docker compose --profile airflow up -d mlflow api prometheus grafana airflow
    soc-ui
.\.venv\Scripts\python.exe scripts\replay_ciciot2023_traffic.py --rows 500 --
    sleep 0.02 --timeout 30 --retries 2
```

9 Interface SOC et résultats

9.1 Interface Streamlit

L'interface Streamlit transforme les artefacts ML en outil utilisable par un analyste [11]. Elle contient une vue SOC, une file d'événements, un module de live scoring, un onglet preuves modèle et un export de note d'incident.

L'interface a été ajoutée pour montrer la couche utilisateur finale. Un modèle MLOps ne doit pas rester invisible dans un terminal : il doit être exploitable par un profil métier. La file analyste permet de filtrer les événements critiques, le live scoring interroge l'API FastAPI et l'onglet Model Evidence rassemble les métriques, la validation et le drift.

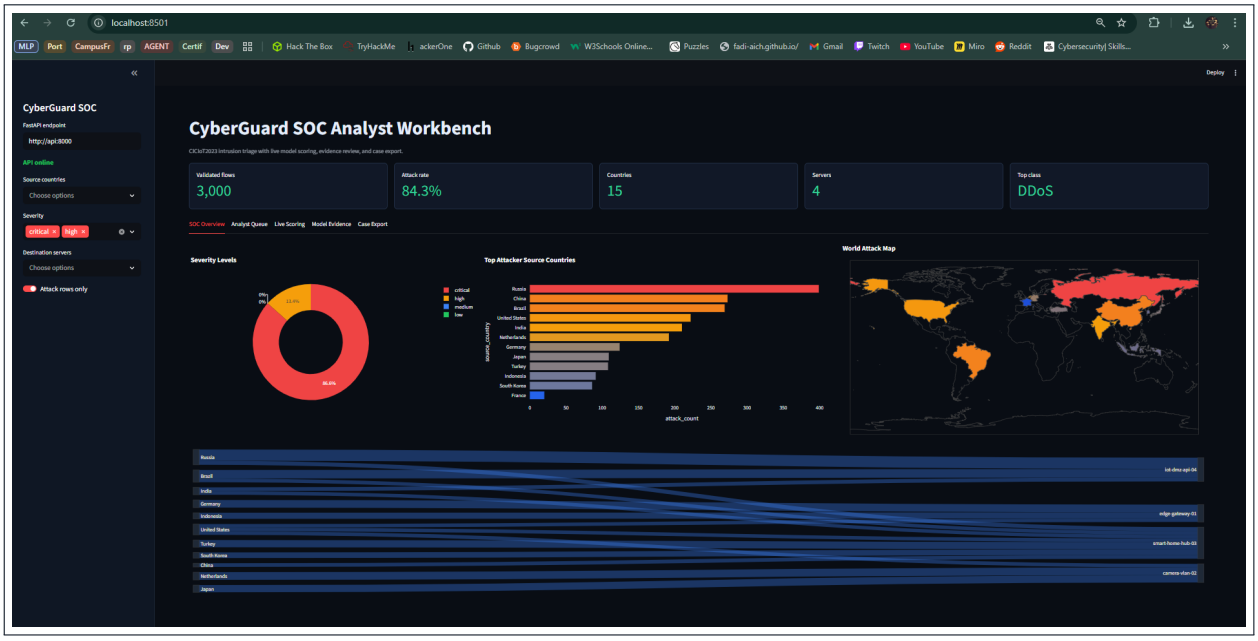

Figure 9.1 – Vue globale CyberGuard SOC Analyst Workbench.

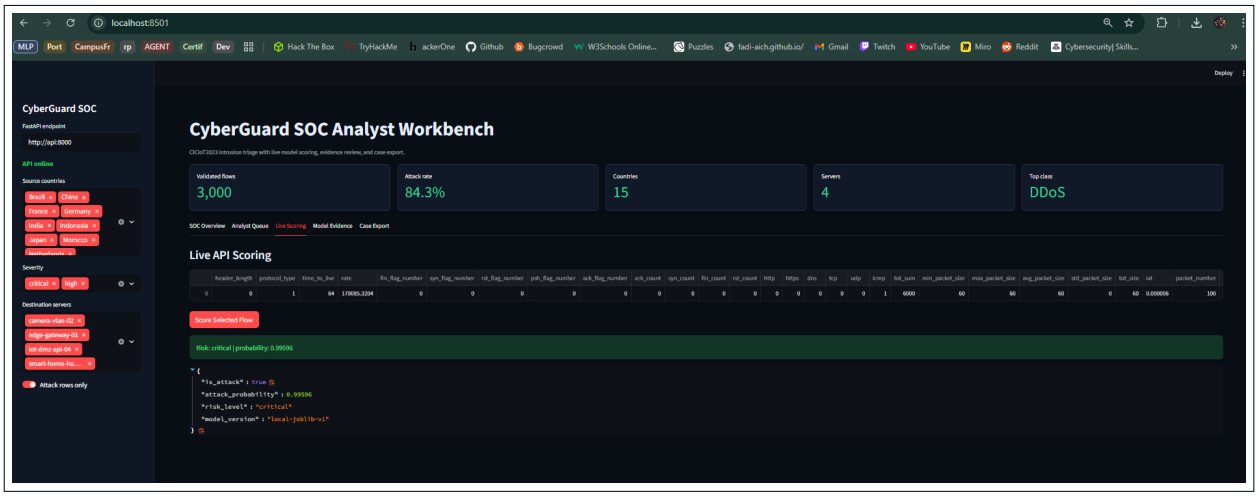

Figure 9.2 – Live scoring et export d'incident.

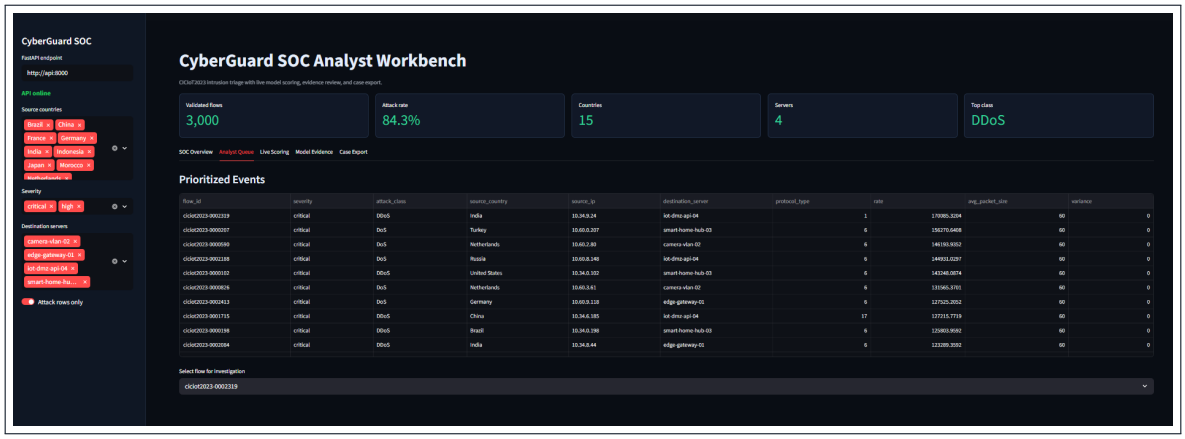

Figure 9.3 – File Analyst Queue.

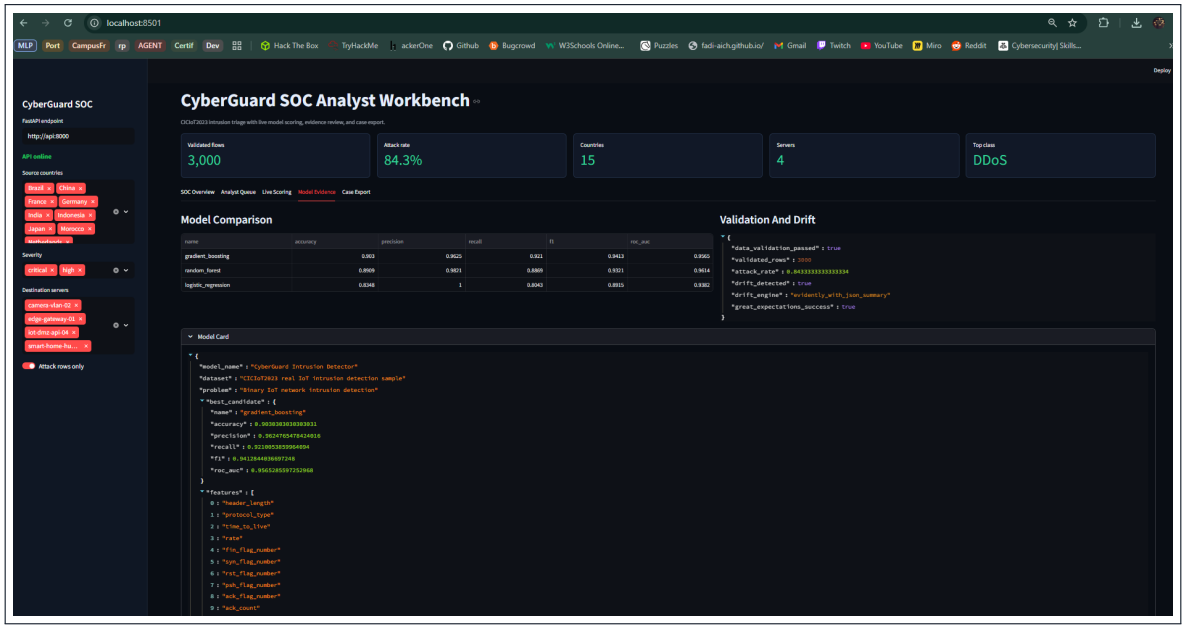

Figure 9.4 – Onglet Model Evidence.

9.2 Structure du projet

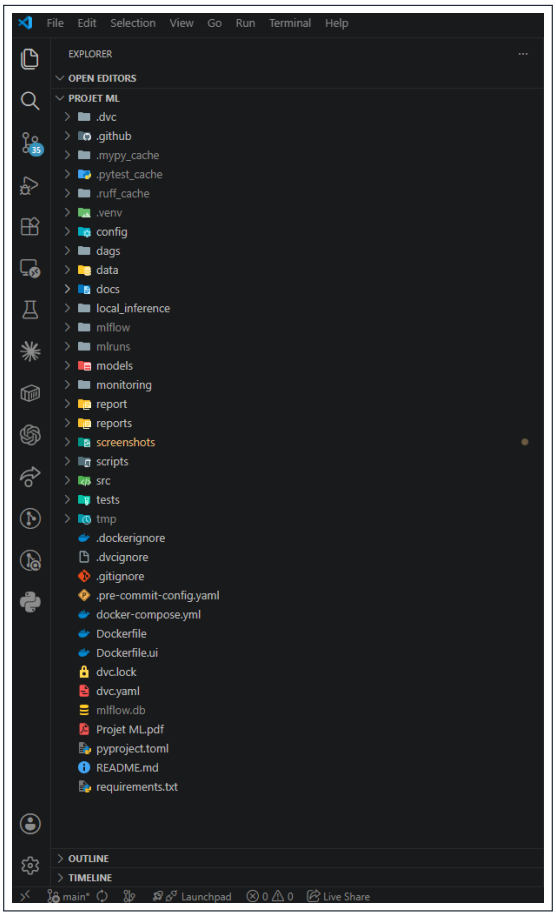

Figure 9.5 – Structure du projet dans VS Code.

9.3 Résultats principaux

Résultat final

Le meilleur modèle est **Gradient Boosting** avec accuracy 0,903, précision 0,962, rappel 0,921, F1-score 0,941 et ROC-AUC 0,956.

Table 9.1 – Synthèse des preuves du rendu

Exigence	**Preuve**
Dataset réel	CICIoT2023, ingestion et validation.
DVC	DAG, metrics show, status.
MLflow	Expériences, run details, registry.
Airflow	DAG success et graph view.
Docker	Docker Desktop et terminal.
API	FastAPI docs et prédiction.
Monitoring	Prometheus targets, Grafana SOC.
Drift	Evidently HTML et résumé JSON.
CI/CD	GitHub Actions success.
UI	Streamlit SOC analyst workbench.

10 Guide de reproduction complet

10.1 Cloner le projet

```
git clone https://github.com/Fadi-AICH/cyberguard-mlops.git
Set-Location -LiteralPath 'C:\Users\fadia\OneDrive\Bureau\Projet ML'
```

10.2 Créer le venv Python 3.11

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\Activate.ps1
$env:PYTHONPATH='src'
```

10.3 Relancer les artefacts ML

```
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.ingest_ciciot2023 --rows
    3000 --page-size 100 --offset 0
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.validate_data
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.great_expectations_check
.\.venv\Scripts\python.exe -m cyberguard_ml.pipeline.train_model
.\.venv\Scripts\python.exe -m cyberguard_ml.monitoring.drift_report
.\.venv\Scripts\python.exe -m cyberguard_ml.monitoring.soc_report
```

10.4 Lancer tous les services

```
docker compose --profile airflow up -d mlflow api prometheus grafana airflow
    soc-ui
docker compose --profile airflow ps
```

10.5 Générer du trafic

```
.\.venv\Scripts\python.exe scripts\replay_ciciot2023_traffic.py --rows 500 --
    sleep 0.02 --timeout 30 --retries 2
```

10.6 URLs

Table 10.1 – URLs de démonstration

Composant	URL
FastAPI	http://localhost:8000/docs
MLflow	http://localhost:5001
Prometheus	http://localhost:9090/targets
Grafana	http://localhost:3000/d/cyberguard-soc/cyberguard-ciciot2023-soc-command-center
Airflow	http://localhost:8080
Streamlit	http://localhost:8501
GitHub Actions	https://github.com/Fadi-AICH/cyberguard-mlops/actions

10.7 Compilation du rapport

```
Set-Location -LiteralPath 'C:\Users\fadia\OneDrive\Bureau\Projet ML\repport'
pdflatex -interaction=nonstopmode -halt-on-error main.tex
biber main
pdflatex -interaction=nonstopmode -halt-on-error main.tex
pdflatex -interaction=nonstopmode -halt-on-error main.tex
```

11 Conclusion

CyberGuard MLOps démontre une chaîne complète de Machine Learning Operations appliquée à la cybersécurité IoT. Le projet répond aux exigences du sujet et va plus loin avec une interface analyste SOC, un dashboard Grafana avancé, un rapport Evidently, un registre MLflow et une CI/CD fonctionnelle.

11.1 Bilan

Les points forts sont :

- dataset réel CICIoT2023 ;
- pipeline DVC reproductible ;
- validation data et Great Expectations ;
- comparaison de plusieurs modèles ;
- tracking et registry MLflow ;
- API FastAPI conteneurisée ;
- monitoring Prometheus/Grafana ;
- Airflow DAG ;
- GitHub Actions ;
- Streamlit SOC UI.

11.2 Difficultés

Les principales difficultés rencontrées sont :

- port MLflow 5000 occupé sous Windows, corrigé avec le port 5001 ;
- dashboard Grafana initial trop simple, reconstruit en SOC command center ;
- URL API dans Streamlit Docker, corrigée avec `http://api:8000` ;
- besoin de rejouer du trafic pour remplir Grafana ;
- gestion des screenshots et de la compilation LaTeX.

11.3 Perspectives

Pour aller encore plus loin, on pourrait ajouter Kubernetes, un vrai feature store, une base de feedback analyste, une promotion automatique en registry, des tests de charge et un monitoring de concept drift avec labels de production.

A Inventaire des captures

Table A.1 – Captures intégrées au rapport

No	Fichier	Preuve
1	`01_environment_versions.png`	Versions environnement.
2	`02_ciciot2023_ingestion_validation.png`	Ingestion et validation.
3	`03_training_metrics_ciciot2023.png`	Entraînement.
4	`04_dvc_pipeline_ciciot2023.png`	Pipeline DVC.
5	`05_quality_tests.png`	Qualité et tests.
6	`06_mlflow_registry_ciciot2023.png`	MLflow registry.
7	`07_fastapi_docs_ciciot2023.png`	FastAPI docs.
8	`08_prediction_response.png`	Réponse prédiction.
9	`09_docker_services-desktop.png`	Docker Desktop.
10	`09_docker_services-terminal.png`	Docker terminal.
11	`10_prometheus_targets.png`	Prometheus.
12	`11_grafana_soc_dashboard.png`	Grafana SOC.
13	`12_evidently_drift_html.png`	Evidently drift.
14	`13_soc_threat_report_html.png`	Rapport SOC.
15	`14_airflow_dag_success.png`	Airflow success.
16	`15_github_repo.png`	GitHub repo.
17	`15_github_repo_actions.png`	GitHub Actions.
18	`16_streamlit_soc_analyst_ui.png`	Streamlit overview.
19	`17_streamlit_live_scoring_case_export.png`	Live scoring.
20	`18_streamlit_AnalystQueue.png`	Analyst queue.

No	Fichier	Preuve
21	`19_streamlit_ModelEvidence.png`	Model evidence.
22	`20_Vscode_structure.png`	Structure code.
23	`21_airflow_dag_graph_view.png`	Airflow graph.
24	`22_mlflow_run_details_metrics_artifacts.png`	MLflow run details.

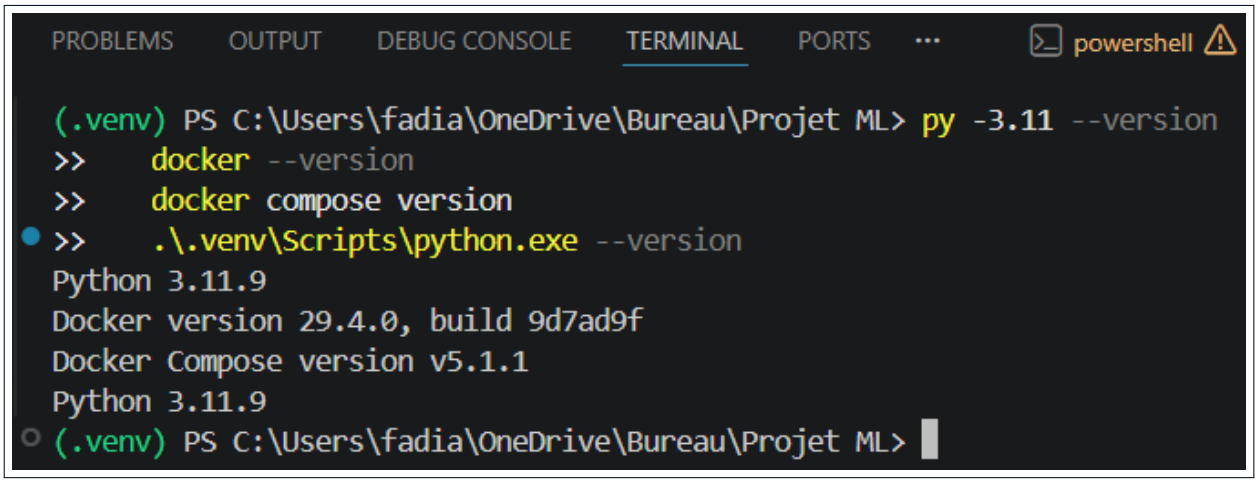

Figure A.1 – Versions principales de l'environnement.

B Commandes détaillées du projet

B.1 Vérification environnement

```
python --version
py -3.11 --version
docker --version
docker compose version
git --version
node --version
npm --version
```

B.2 Arrêt et redémarrage

```
docker compose --profile airflow down
docker compose --profile airflow up -d mlflow api prometheus grafana airflow
    soc-ui
docker compose --profile airflow ps
```

B.3 Tests API

```
Invoke-RestMethod http://localhost:8000/health
Invoke-WebRequest http://localhost:8000/docs
Invoke-WebRequest http://localhost:8000/metrics
```

B.4 GitHub

```
git status
git add -A
git commit -m "Build CyberGuard MLOps project"
git push origin main
gh workflow run ci.yml --repo Fadi-AICH/cyberguard-mlops
```

C Références

[1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu et A. A. Ghorbani, « CICIoT2023 : A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment, » *Sensors*, t. 23, n° 13, p. 5941, 2023. DOI : 10.3390/s23135941 adresse : https://www.mdpi.com/1424-8220/23/13/5941

[2] Canadian Institute for Cybersecurity, University of New Brunswick, *CIC IoT Dataset 2023*, Consulté en mai 2026, 2023. adresse : https://www.unb.ca/cic/datasets/iotdataset-2023.html

[3] Data Version Control, *Defining Pipelines*, Consulté en mai 2026, 2026. adresse : https://dvc.org/doc/user-guide/pipelines/defining-pipelines

[4] Great Expectations, *Data Validation Workflow*, Consulté en mai 2026, 2026. adresse : https://docs.greatexpectations.io/docs/guides/validation/validate_data_overview/

[5] MLflow, *MLflow Tracking Documentation*, Consulté en mai 2026, 2026. adresse : https://www.mlflow.org/docs/latest/ml/tracking

[6] Apache Airflow, *Tasks and Dependencies*, Consulté en mai 2026, 2026. adresse : https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/tasks.html

[7] GitHub, *Workflow Syntax for GitHub Actions*, Consulté en mai 2026, 2026. adresse : https://docs.github.com/actions/learn-github-actions/workflow-syntax-for-github-actions

[8] Prometheus, *Prometheus Python Client Instrumenting*, Consulté en mai 2026, 2026. adresse : https://prometheus.github.io/client_python/instrumenting/

[9] Grafana Labs, *Grafana Dashboard Best Practices*, Consulté en mai 2026, 2026. adresse : https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/best-practices/

[10] Evidently AI, *Data Drift Preset*, Consulté en mai 2026, 2026. adresse : https://docs.evidentlyai.com/metrics/preset_data_drift

[11] Streamlit, *Create an App*, Consulté en mai 2026, 2026. adresse : https://docs.streamlit.io/get-started/tutorials/create-an-app